2024

Глава 6 Работа с файлами

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Лектор: д.т.н., проф.
Иванова Галина Сергеевна

6.1 Файловая система

Файл – поименованная последовательность элементов данных (компонентов файла), хранящихся, как правило, во внешней памяти.

Как исключение данные файла могут не храниться, а вводиться с внешних устройств (ВУ), например клавиатуры или выводиться на ВУ, например экран.

Windows Синтаксис имени файла:

[Имя_диска:[Список_имен_каталогов\]] Имя_файла.Расширение

Имя файла в Windows составляют из строчных и прописных букв латинского и русского алфавитов, арабских цифр и некоторых специальных символов, например, символов подчеркивания «_» или доллара «$»

Расширение определяет тип хранящихся данных, например:

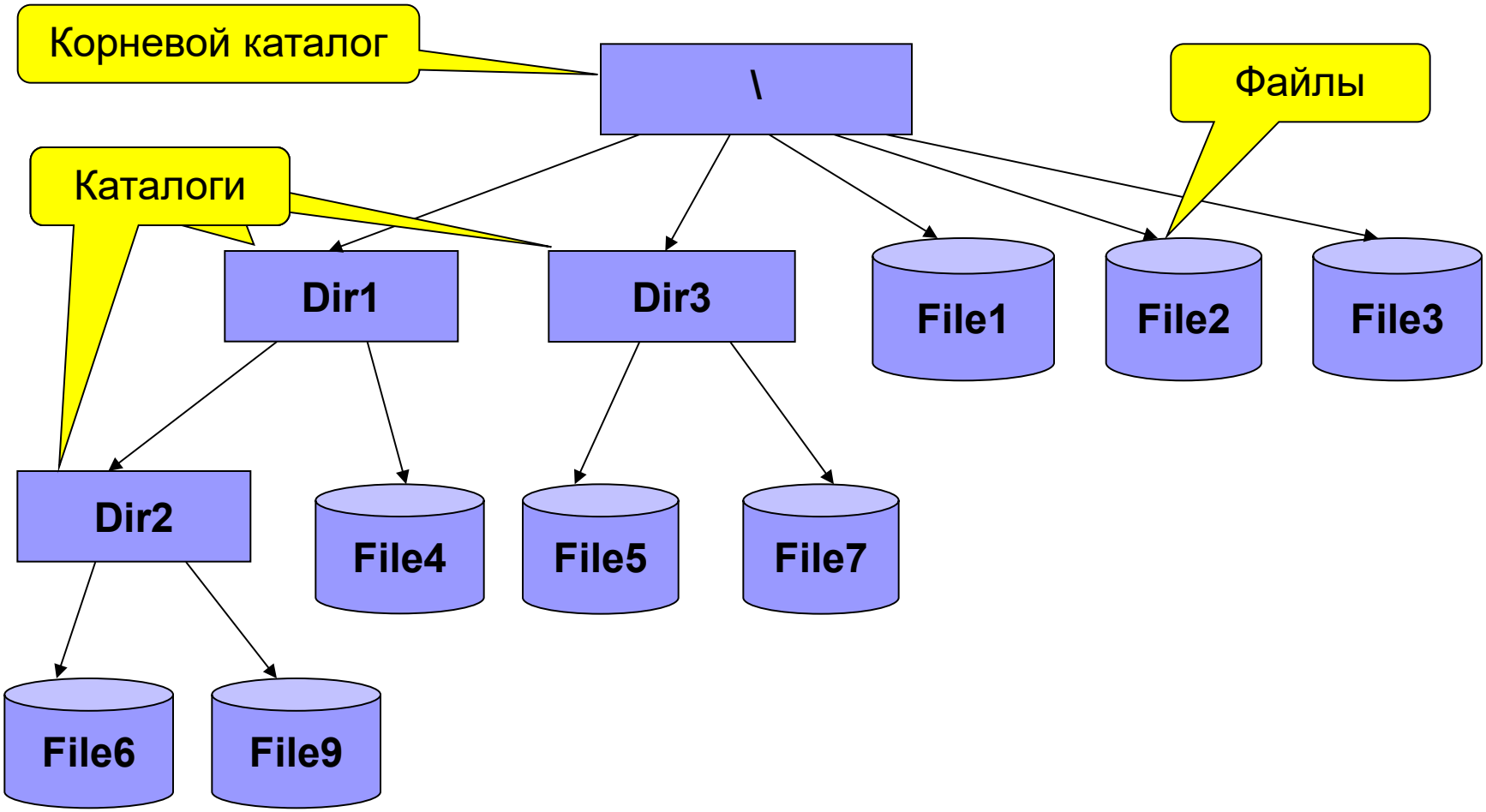
COM, EXE – исполняемые файлы (программы);

PAS, BAS, CPP – исходные тексты программ на алгоритмических языках ПАСКАЛЬ, БЭЙСИК и С++;

BMP, JPG, PIC – графические файлы (рисунки, фотографии);

WAV,MP3,WMA – аудиофайлы.

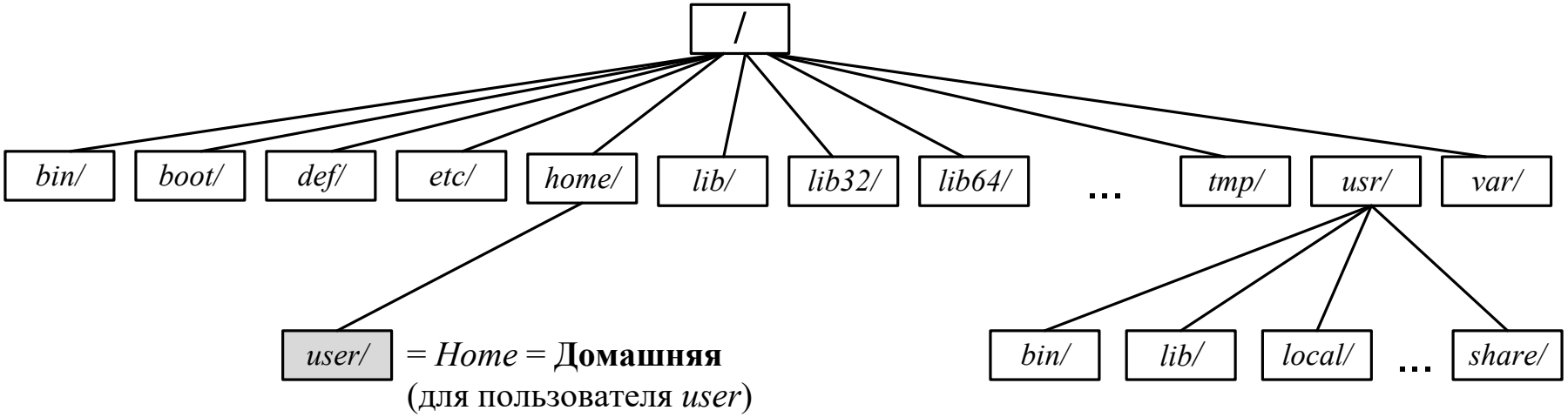
Организация файлов на внешнем носителе в Windows



Пример полного имени файла:

`D:\Dir1\Dir2\File9.`

Организация файлов на внешнем носителе в ОС Linux



Абсолютное имя файла: **`/home/user/data.txt`**.

Примеры:

`/` – корневой каталог *Linux*;

`~/` – домашний каталог **`/home/user`** пользователя *user Linux*;

`../` – родительский каталог для текущего каталога;

`~/progs/data.txt` – файл в подкаталоге progs домашнего каталога ;

`../data.txt` – файл в родительском каталоге;

`data.txt` – файл в текущим каталоге;

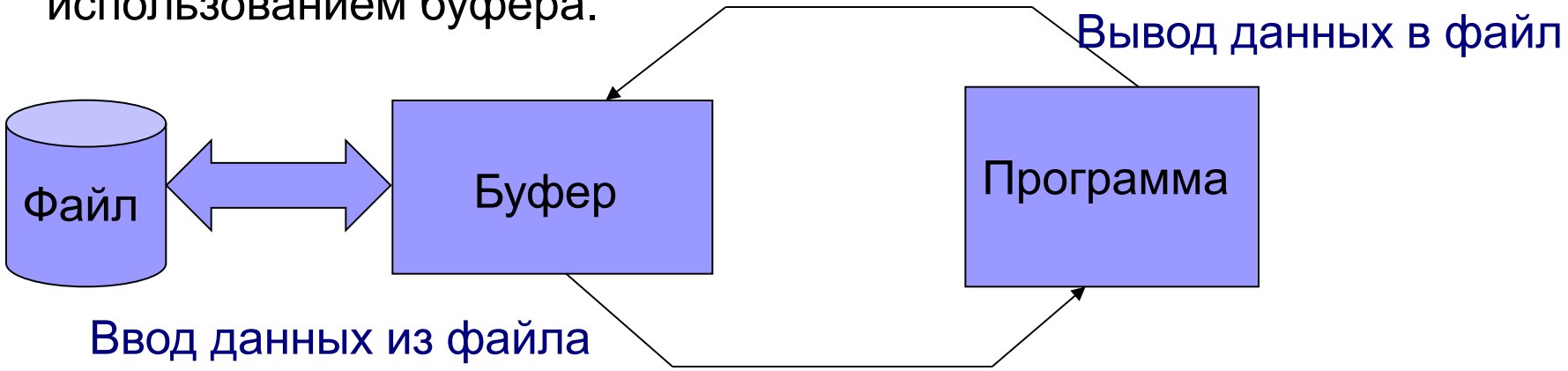
`./prog` – вызов программы из текущего каталога.

Файлы в С++

Файл *с точки зрения языка С++* – последовательность однотипных компонентов: файл записей (структур), файл целых чисел, файл строк. В зависимости от типа компонентов различают два типа файлов: *текстовые и двоичные*.

Количество компонентов файла при объявлении файловой переменной не указывается. Максимальный размер файла определяется свободным пространством на устройстве, например, диске.

Физически операции ввода-вывода с файлами выполняются с использованием буфера.

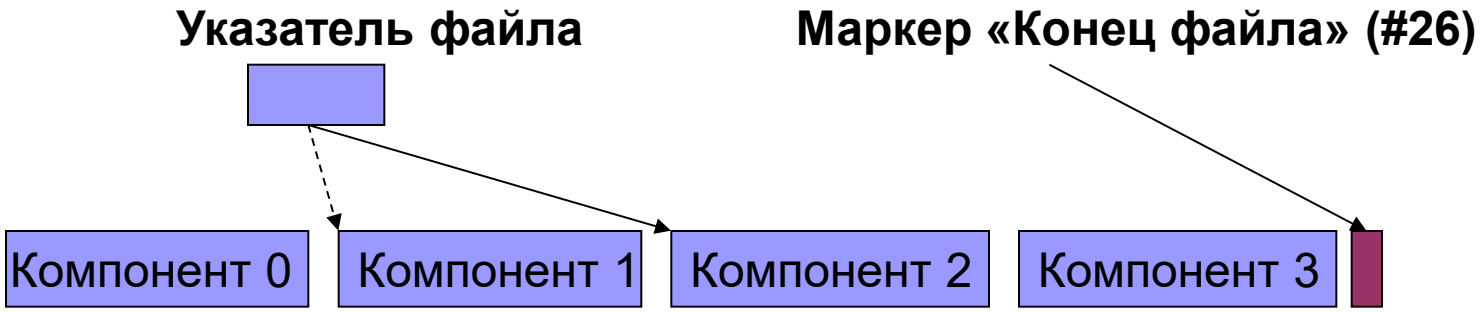


Для файлов принципиально возможен не только последовательный, но и произвольный доступ, при котором чтение информации осуществляется по рассчитанному по номеру компонента и его длине смещению.

Файловый указатель и файловая переменная

Доступ к компонентам файла осуществляется через ***файловый указатель (или указатель файла)***.

При выполнении операции чтения или записи указатель автоматически перемещается на следующий компонент.



После вывода последнего компонента файла система пишет специальную запись – маркер «Конец файла» (байт #26).

При обнаружении во время операции чтения маркера конца файла – операция завершается. Попытка читать маркер вызывает прерывание по ошибке чтения.

Для работы с файлами используются ***файловые переменные*** – указатели типа FILE. Структура FILE - управляющая таблица для организации доступа к файлу.

6.2 Объявление, открытие и закрытие файлов

Синтаксис объявления файловой переменной (Ф.п.):

FILE *Файловая_переменная; // объявление указателя
// на управляющую таблицу FILE

Открытие файла

Ф.п. = fopen(Имя_файла,Операция [+] [Тип]); // выделение памяти и
// заполнение управляющей таблицы

Операция[+]:

r - ввод из существующего файла;
w - вывод с очисткой файла или создание нового файла для вывода;
a - добавление к существующему или создание файла для вывода;
r+ - ввод/вывод в существующий файл;
w+ - ввод/вывод в существующий или создание нового файла;
a+ - ввод/добавление к существующему или создание файла для ввода/вывода.

Тип - t - текстовый файл (принимается по умолчанию);
b - двоичный файл.

Закрытие файла:

fclose(Ф.п.);

Примеры открытия/закрытия файлов

а) объявление и открытие существующего или нового двоичного файла для ввода/вывода

```
FILE *f;
f=fopen("abc.txt","w+b");
 …
fclose(f);
```

б) объявление и открытие существующего файла с проверкой существования под *Windows*:

```
FILE *f;
if ((f=fopen("f:\\iva\\text.txt","r"))!=nullptr) …
   …
fclose(f);
```

в) объявление и открытие существующего файла под Linux:

```
FILE *f=fopen("//home//user//projects//data.txt","r");
```

6.3 Функции управления файловым указателем

Файловый указатель содержит смещение в байтах относительно начала файла.

а) определение положения файлового указателя:

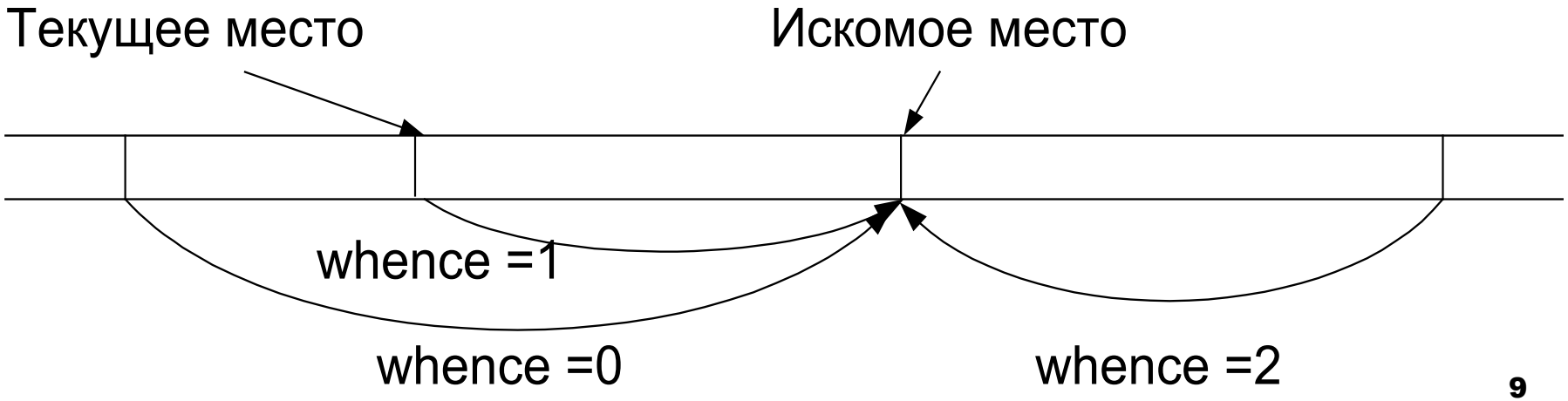
```
long ftell(FILE *stream);
```

б) установка файлового указателя на начало файла:

```
int rewind(FILE *stream);
```

в) установка файлового указателя в произвольное место:

```
int fseek(FILE *stream,long offset,int whence);
```



Обработка компонентов файла

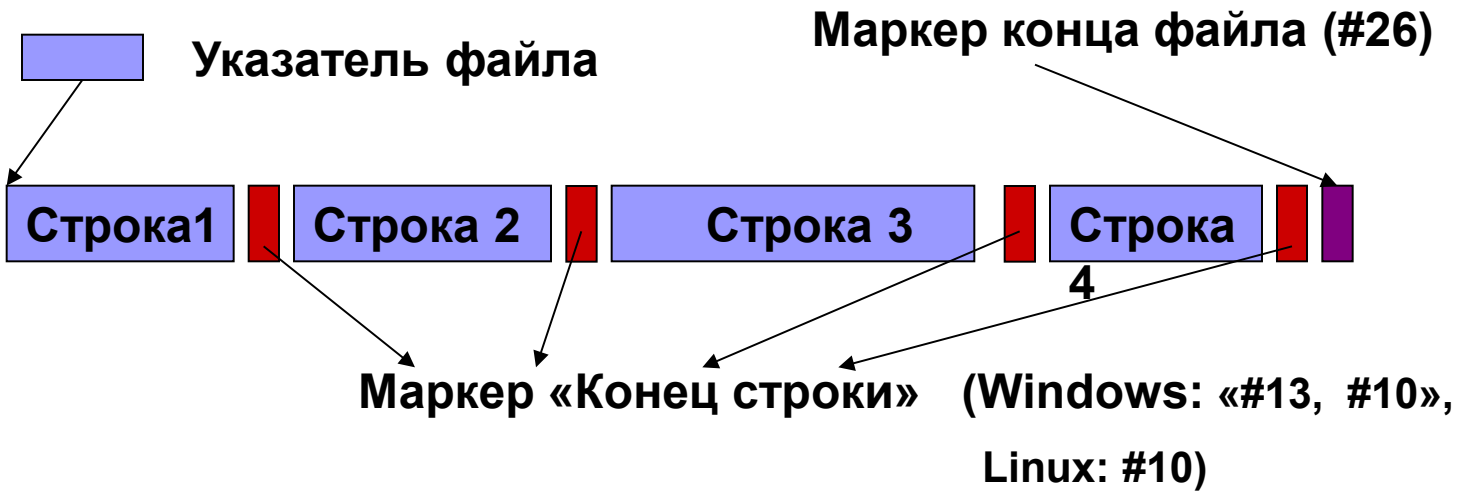
Основные операции над компонентами – операции записи и чтения. На базе этих операций выполняют более сложные операции:

- *создание файла* – занесение в файл требуемых записей;
- *модификация файла* – изменение всех или нескольких записей, добавление и удаление записей;
- *поиск* нужной информации в файле.

Операции записи и чтения для каждого типа файла осуществляется по-своему.

6.4 Текстовые файлы

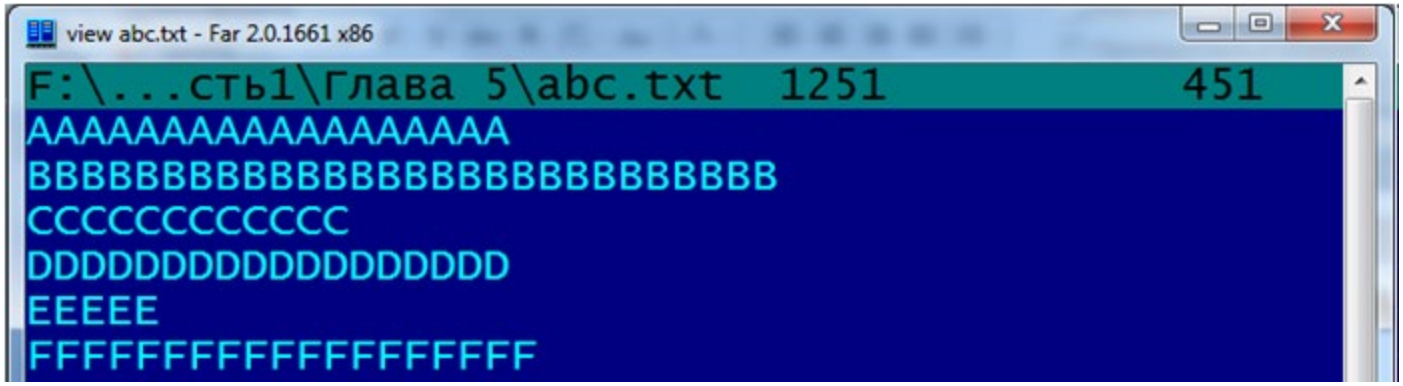
Текстовый файл – файл, компонентами которого являются символьные строки переменной длины, заканчивающиеся специальным маркером – маркером «Конец строки».



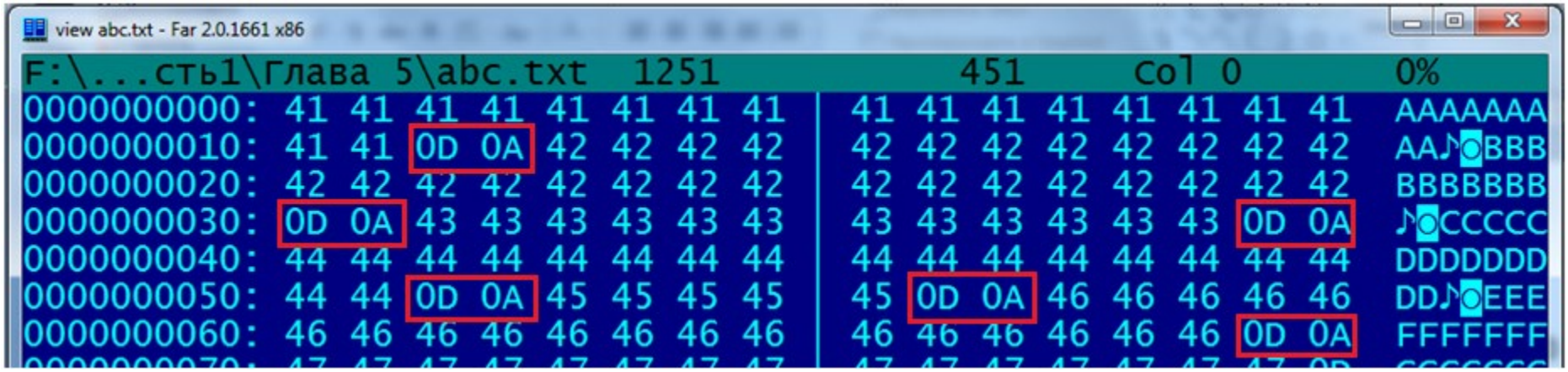
Текстовые файлы используют для хранения и обработки символов, строк, символьных массивов. *Числовые и логические данные при записи в текстовые файлы должны преобразовываться в символьные строки*.

Текстовый файл в Windows

В символьном представлении :

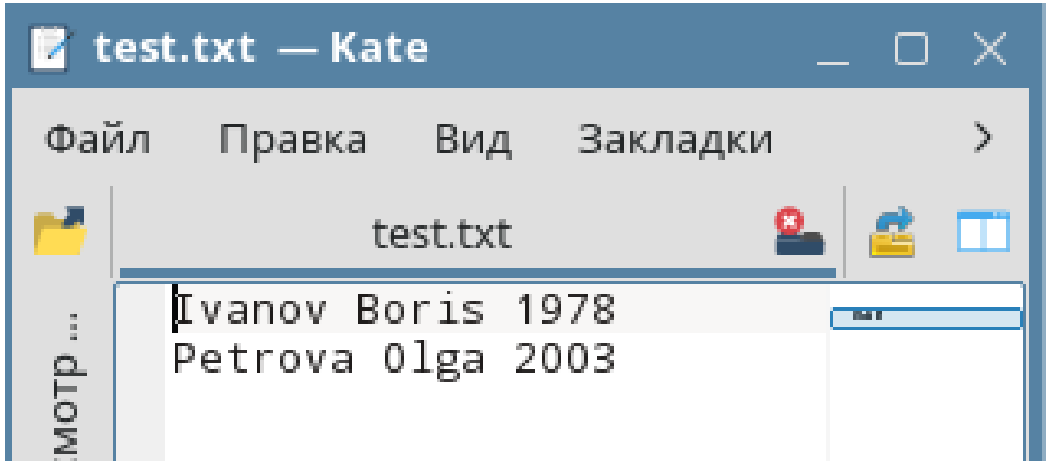


В шестнадцатеричном представлении:

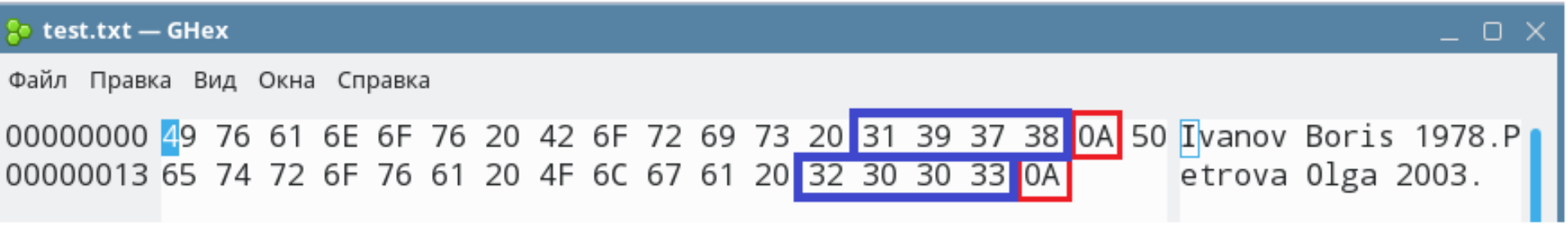


Текстовый файл в Linux

Символьное представление



Шестнадцатеричное представление



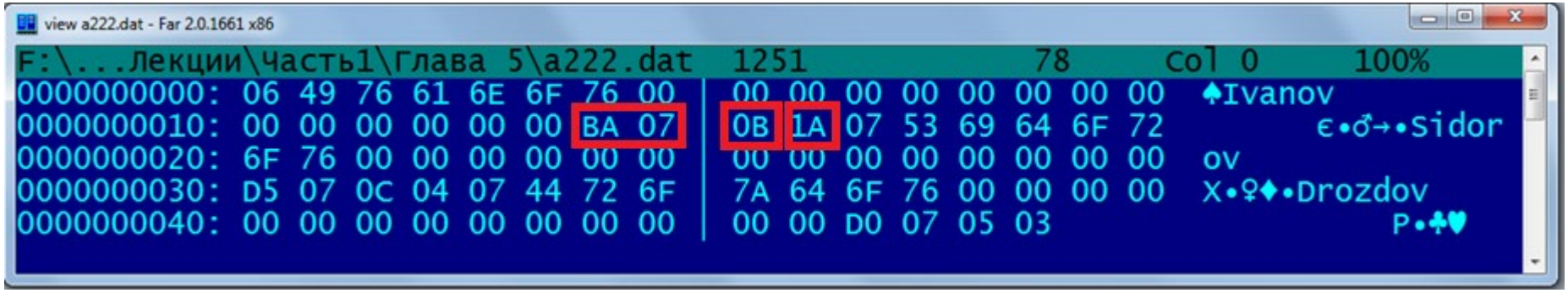
31 39 37 38 $_{16\ ANSI}$ -> 1978_{10}

Двоичный файл

Текстовый редактор прочитал файл некорректно

Символьное представление

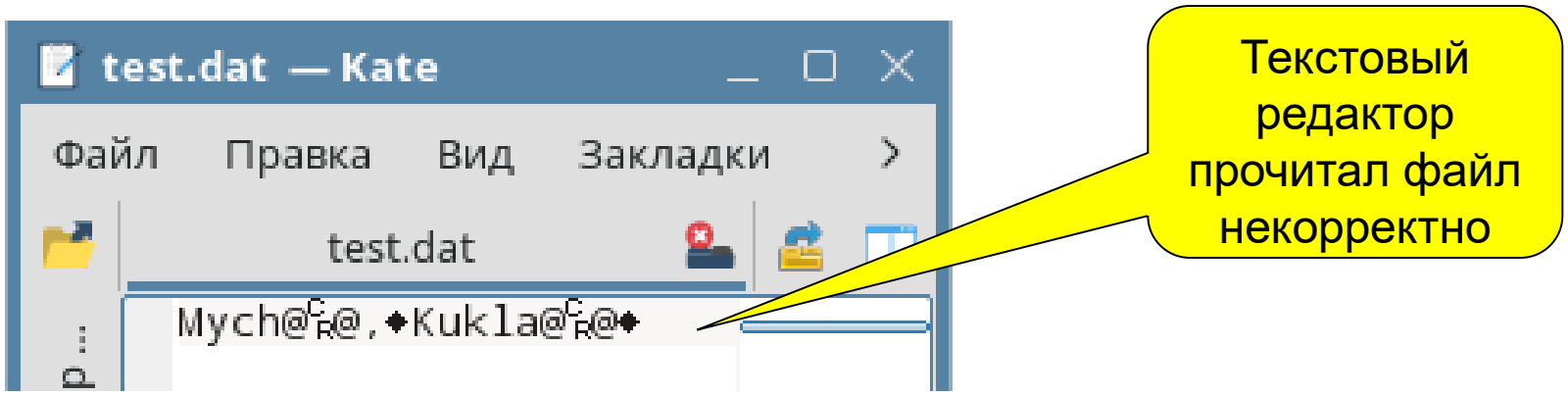
Шестнадцатеричное представление



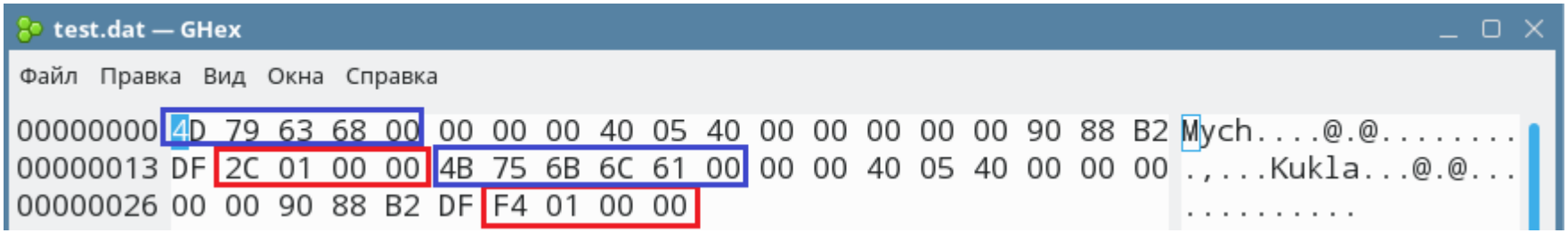
BA 07 -> 07 BA_{16} = ?

Двоичный файл

Символьное представление



Шестнадцатеричное представление



2C 01 00 00 -> 00 00 01 $2C_{16}$ = ?

1-2. Ввод/вывод символов

```
int getc(FILE *stream);  //возвращает символ или EOF(-1)
int putc(int c,FILE *stream);
```

Пример. Вывод на экран содержимого файла, полное имя которого указывается в командной строке, например:

/home/user/projects/Ex06_01/test.txt

```
#include <stdio.h>
int main(int argc,char *argv[]) {
  FILE *in;    int ch;
  if (argc<2)puts("Enter file name.");
  else
    if ((in=fopen(argv[1],"r"))!=nullptr) {
        while ((ch=getc(in))!=EOF) putchar(ch);
        fclose(in);
    }
    else puts("No open file.");
   return 0; }
```

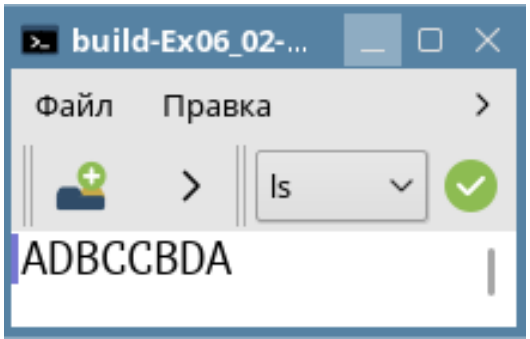
Стандартные текстовые файлы

```
stdin, stdout, stderr
       getchar( )  = = getc(stdin)
       putchar(ch) = = putc(ch,stdout)
```

ABCD
⇓
ADBCCBDA

Пример. Чтение с начала и с конца Ex06_02

```
#include <stdio.h>
int main()
 { FILE *f;  long offset=0L;    int ch;
   f=fopen("//home//user//Ex06_02//test.txt.dat","r");
   while ((!fseek(f,offset++,0)) && ((ch=getc(f))!=EOF))
    { putc(ch,stdout);
      if (!fseek(f,-(offset+1),2)) putc(getc(f),stdout);
    }
   fclose(f);
   return 0;
 }
```



Буферированные и «прямые» операции

stdio.h: **getchar(), putchar()** – буферированные операции;

conio.h: **getch()** – прямой ввод без копирования на экран,

getche() – тоже, но копирует на экран вводимый символ,

putch() – прямой вывод символа на экран.

Примеры:

а) `while((n=getchar())!='E'){putchar('\n'); putchar(n);}`

```
ABCE
↵
A↵
B↵
C
```

б) `while ((n=getche())!='E') {putch('\n'); putch(n); }`

```
A↵
AB↵
BC↵
CE
```

3. Вывод строк

```
int fputs(const char *s,FILE *stream);
```
// не пишет \0!!!

Пример. Создание файла из 6 строк (Ex06_03).

```
#include <stdio.h>
int main()
{    FILE *f;
     const char *s="ABCD";
     f=fopen("test.txt","w");
     for (int i=0;i<6;i++)
       {    fputs(s,f);
            fputs("\n",f); // разделитель
       }
     fclose(f);
     return 0;
}
```

ABCD↵
ABCD↵
ABCD↵
ABCD↵
ABCD↵
ABCD↵

test.txt — GHex

Файл Правка Вид Окна Справка

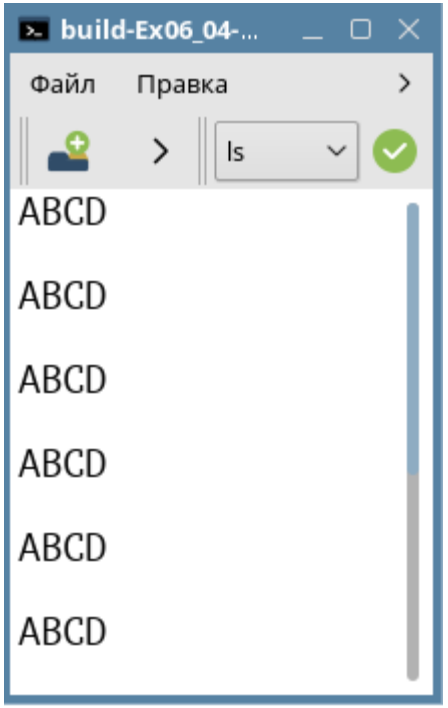
```
00000000 41 42 43 44 0A 41 42 43 44 0A 41 42 43 44 0A 41 42 43 44 ABCD.ABCD.ABCD.ABCD
00000013 0A 41 42 43 44 0A 41 42 43 44 0A                         .ABCD.ABCD.
```

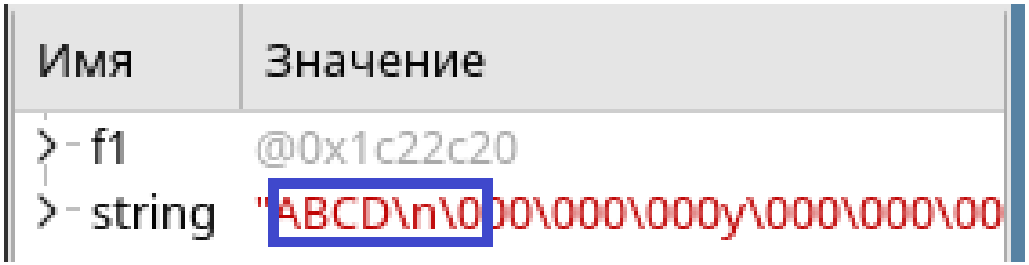
4. Ввод строк

`char *fgets(char *s, int n, FILE *stream);` // возвращает адрес строки или NULL, читает до ‘\n’, включая его!

Пример. Чтение файла по строкам (Ex06_04).

```
#include <stdio.h>
int main()
{
   FILE *f1;
   char string[80];
   f1 = fopen("test.dat", "r");
   while (fgets(string, 80, f1) != nullptr)
       puts(string);
   return 0;
}
```



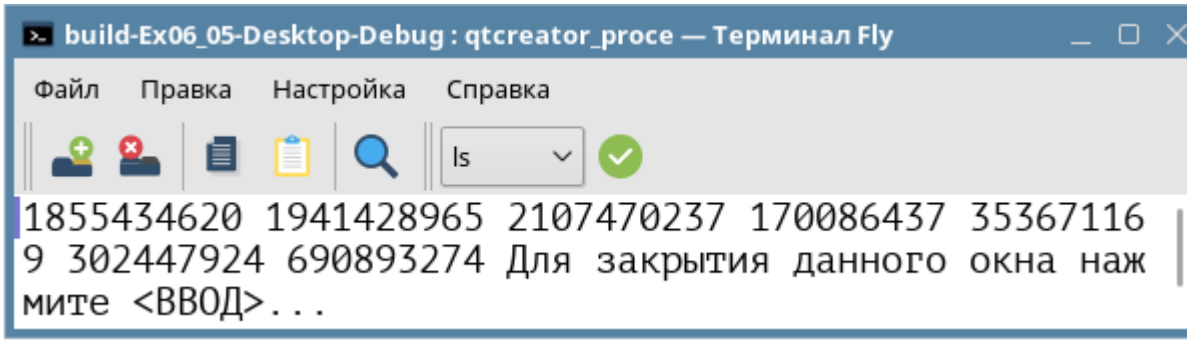


5-6. Форматный ввод/вывод

```
int fscanf(FILE *stream,const char *format[,adress,...]);
int fprintf(FILE *stream,const char *format[,argument,.]);
```

Пример. Создание и распечатка файла чисел (Ex06_05).

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main()
{   int i,r; FILE *f;
    srand(static_cast<unsigned int>(time(nullptr)));
    f = fopen("rand1.txt","w+");
    for (i=0;i<7;i++){ r = rand();
       fprintf(f,"%d ",r);}
    rewind(f);
    while (fscanf(f,"%d",&r)!=EOF) printf("%6d ",r);
    fclose(f);
    return 0;
}
```

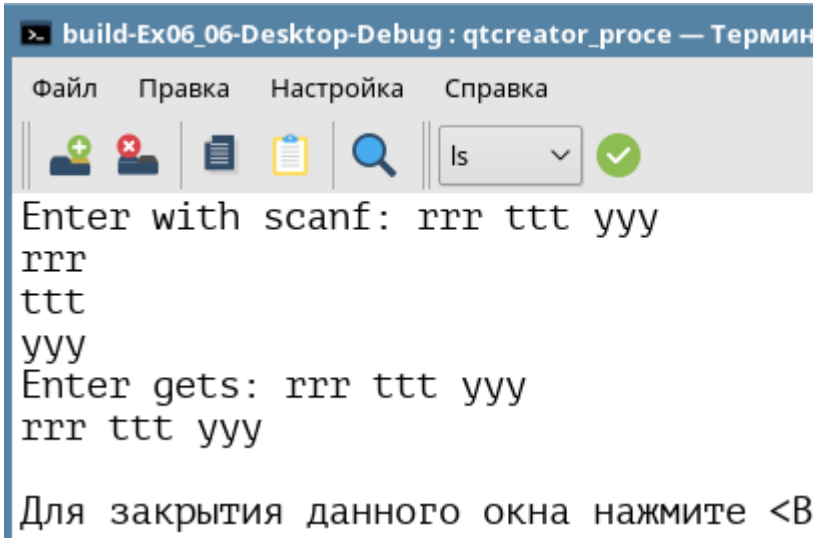


Очистка буфера ввода (Ex06_06)

Пример:

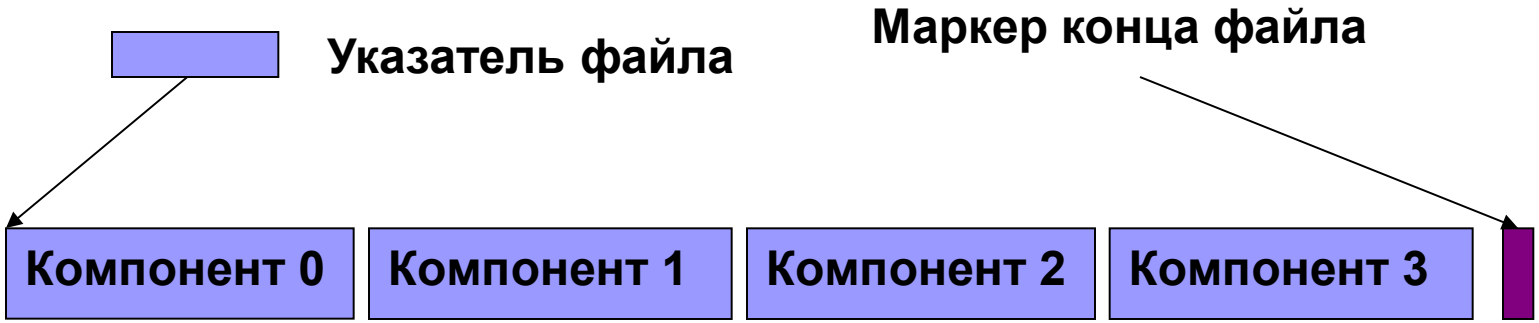
```
#include <stdio.h>

int main() {
    int i;
    char string[81];
    printf("Enter with scanf: ");
    for(i = 0; i<3; i++){
        scanf("%s",string); // не вводит Enter!
        printf("%s\n",string);
    }
    // Необходимо освободить буфер, содержащий код Enter !!!
    while (getchar() != '\n');
    printf("Enter gets: " );
    fgets(string,80,stdin);  // gets - исключена из стандарта
    printf("%s\n", string );
}
```



4.5 Двоичные файлы

Двоичные файлы хранятся на диске во внутреннем представлении и могут состоять из компонентов одного или различных типов (журналы). В первом случае все компоненты одного размера:



Все компоненты одного размера

Типизированный файл можно открыть для записи и чтения. Файл, открытый для записи, может использоваться для чтения. В файл, открытый для чтения, можно писать.

Функции чтения-записи

Ввод/вывод

```
size_t fread(void *ptr,size_t size,size_t n,
                                        FILE *stream);
size_t fwrite(void *ptr,size_t size,size_t n,
                                        FILE *stream);
```

Используются в двух вариантах:

а) при работе со структурами

```
fread  (&myrec, sizeof(myrec),1,f1);
fwrite (&myrec, sizeof(myrec),1,f1);
```

где **myrec** - переменная типа «структура»;

б) при работе с разнотипной информацией

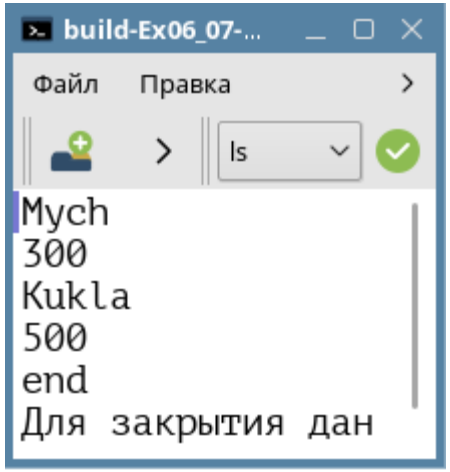
```
fread(&buffer,1,sizeof(buffer),f2);
fwrite(&buffer,1,sizeof(buffer),f2);
```

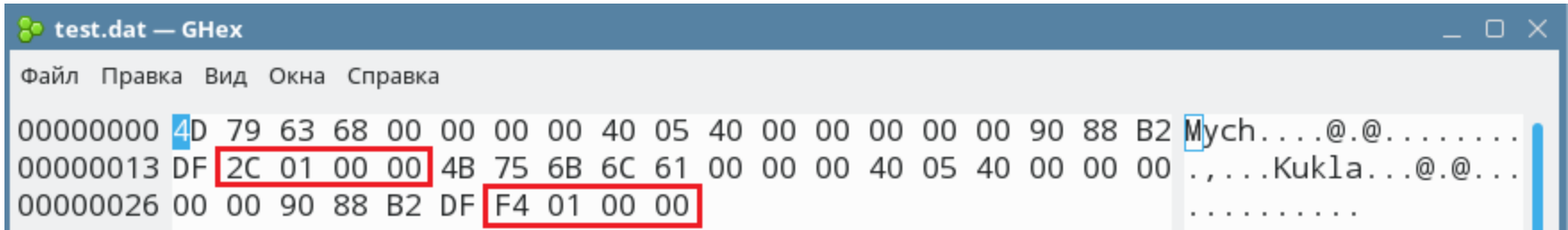
где **buffer** – область памяти, отводимая для хранения и первичной обработки считанных данных, часто это "массив байтов".

Пример работы с двоичным файлом (Ex06_07)

Создание файла записей "Название игрушки, стоимость“.

```
#include <stdio.h>
#include <string.h>
struct toys{ char name[20];int cost;};
int main() {
   FILE *f;    toys toy;
   f=fopen("test.dat","w+b");
   while(scanf("\n%s",toy.name),
         strcmp(toy.name,"end")!=0) {
     scanf("%d",&toy.cost);
     fwrite(&toy,sizeof(toy),1,f);
    }
   fclose(f);
}
```

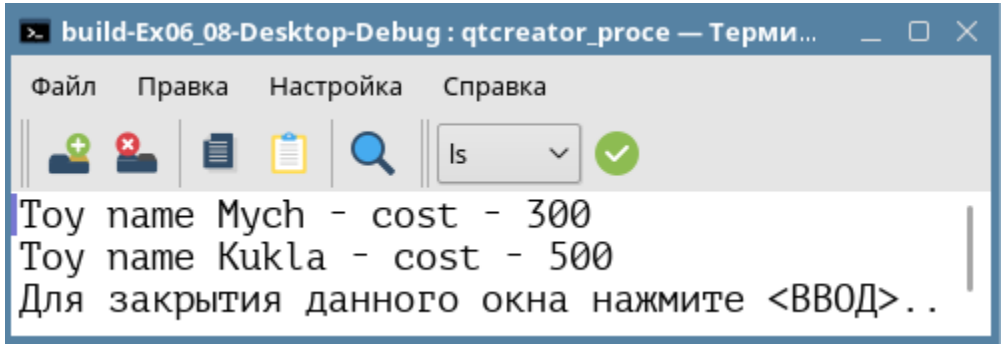




Пример работы с двоичным файлом (Ex06_08)

Чтение и вывод на экран файла записей "Название игрушки, стоимость«.

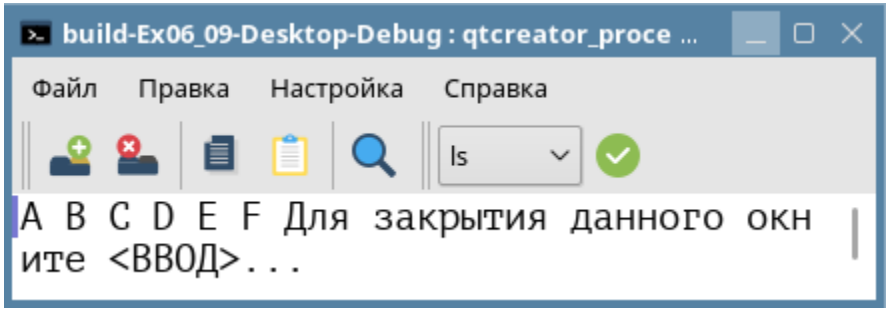
```
#include <stdio.h>
struct toys{ char name[20]; int cost;};
int main()
{
  FILE *f;
  toys toy;
  f=fopen("test.dat","r+b");
  while(fread(&toy,sizeof(toy),1,f)>0)
   printf("Toy name %s - cost - %d\n",toy.name,toy.cost);
  fclose(f);
  return 0;
}
```



Чтение текстового файла как двоичного

Создание текстового файла и чтение его как файла символов (Ex06_09)

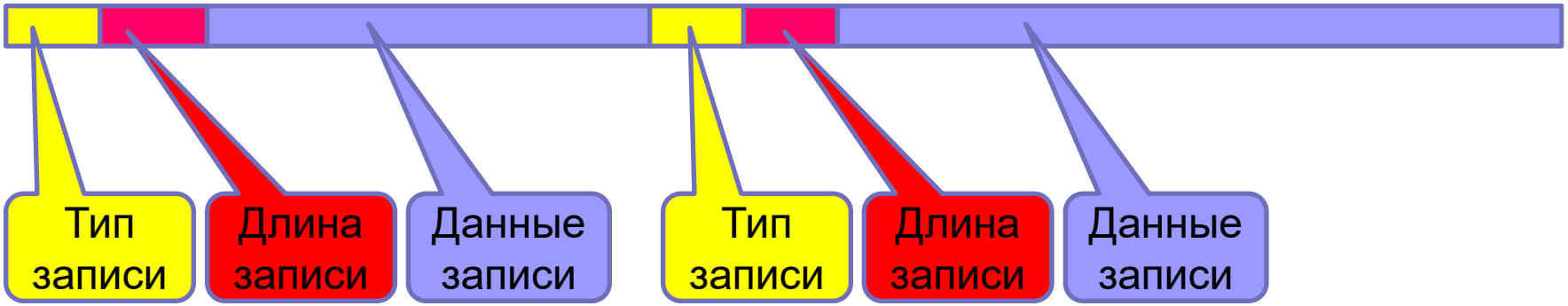
```
#include <stdio.h>
int main()
{  char c;
   FILE *f;
   f=fopen("ddd.dat","w");
   fputs("ABCDEF",f);
   fclose(f);
   f=fopen("ddd.dat","rb");
   while (fread(&c,1,1,f)!=0)
      printf("%c ",c);
   fclose(f);
   return 0;
}
```



Работа с файлами переменной структуры

Чаще всего приходится работать с "журнальными" файлами, размер записи и тип сохраняемой информации в этом случае может быть различен.

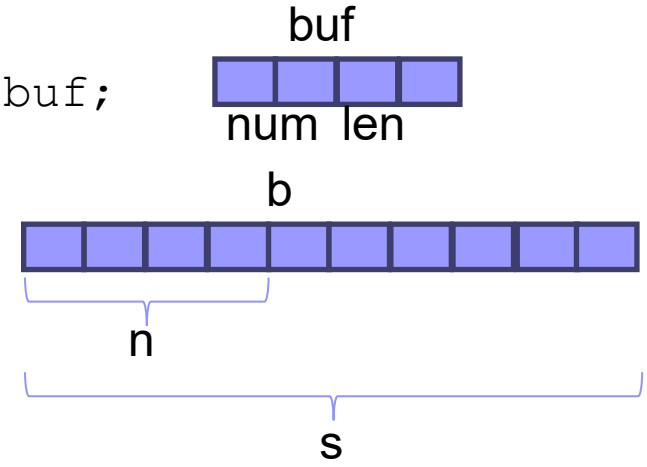
Пример:

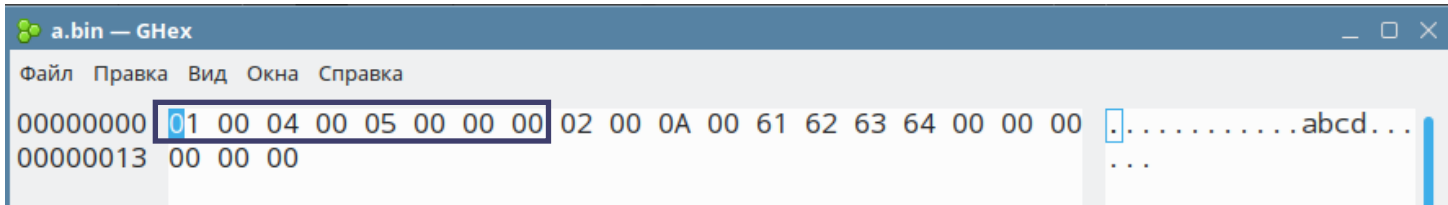


В этом случае сначала читаются два первых поля, а потом – непосредственно данные…

Создание и обработка файла журнального типа

```
#include <stdio.h>
int main(){
    struct base {unsigned short num, len;} buf;
    union body{ int n; char s[10]; } b;
    FILE *f;
    int n = 5;
    char s[10] = "abcd";
    f = fopen("a.bin","wb");
    buf.num = 1;
    buf.len = sizeof(n);
    fwrite(&buf,sizeof(buf),1,f);   // запись заголовка
    fwrite(&n,sizeof(n),1,f);       // запись информационной части
    buf.num = 2;
    buf.len = sizeof(s);
    fwrite(&buf,sizeof(buf),1,f);   // запись заголовка
    fwrite(s,sizeof(s),1,f);        // запись информационной части
    fclose(f);
```

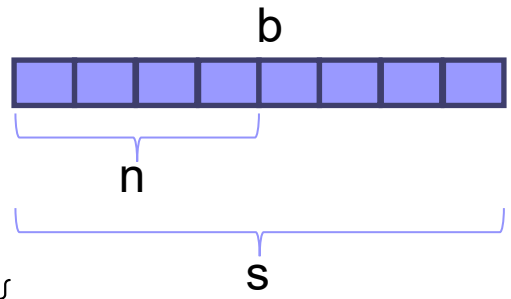


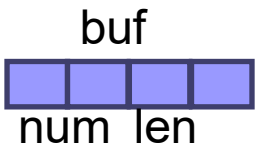


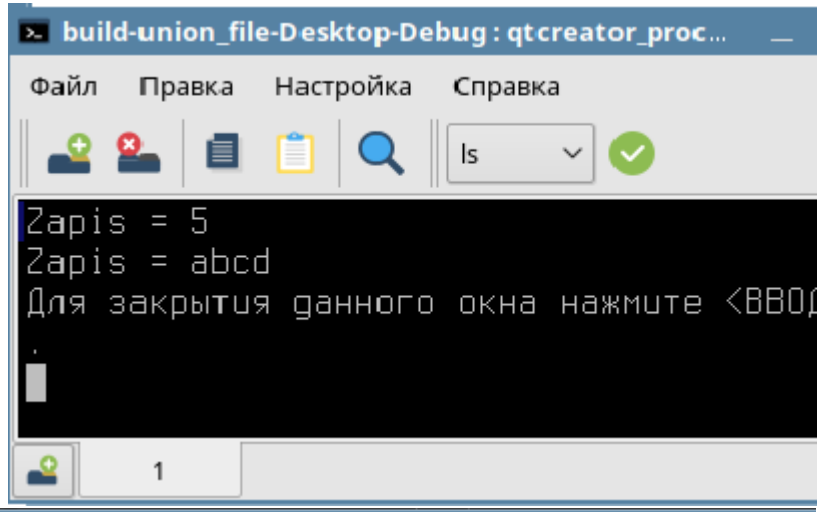
Создание и обработка файла журнального типа

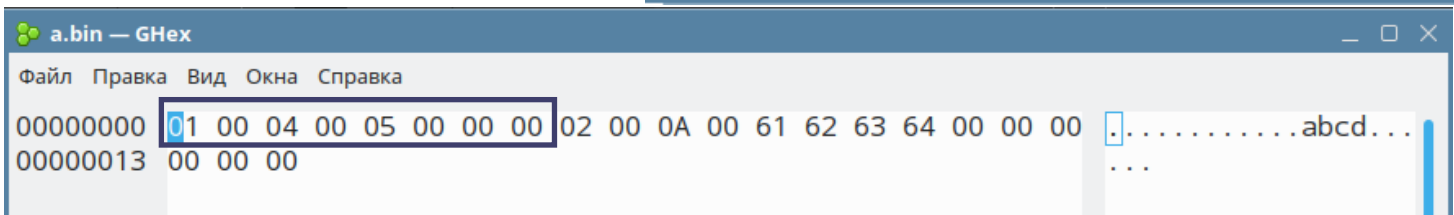
```
//union body{ int n; char s[10]; } b;

  f = fopen("a.bin","rb");
   while(fread(&buf,sizeof(buf),1,f)!=0){
      fread(&b,buf.len,1,f);
      printf("Zapis = ");
      switch(buf.num){
      case 1: printf("%d",b.n);
              break;
      case 2: printf("%s",b.s);
      }
      puts("");
   }
   return 0;
}
```









6.6 Переименование и удаление файлов

Удаление файлов:

```
int unlink(const char *filename);   // возвращает 0, а при неудаче -1

int remove(const char *filename); /*возвращает 0, а при неудаче - не
                                                                ноль */
```

Переименование файлов:

```
int rename(const char *oldname, const char *newname);
```

6.6 Переименование и удаление файлов

Пример. Вставка 10 чисел после первых 10 чисел файла (Ex06_10)

```
#include <stdio.h>
int main()
{  int n,m;
   FILE *f,*g;
   f=fopen("rand.dat","r");
   g=fopen("$$$$xxx.tmp","w");
   for (n=0;n<10;n++)                    // переписывание первых 10 чисел
       {  fscanf(f,"%d\n",&m); fprintf(g,"%d\n",m);}
   for (n=0;n<10;n++) fprintf(g,"%d\n",n); // вставка 10 чисел
   n=fgetc(f);                           // фрагмент дописывание остальных
   while(n!=EOF) { fputc(n,g);  n=fgetc(f); }
   fclose(f); fclose(q);
   remove("rand.dat");                   // удаление файла
   rename("$$$$xxx.tmp","rand.dat");     // переименование файла
}
```